

Task 8 — : Build an S3-Based Static Website and Storage Workflow

Intern: Nidhi Pandey

Program: Evolixa AWS Cloud Internship

Date: 2026-09-05

1. Approach and Architecture

The objective of this task was to host a static website using Amazon S3 while understanding the security implications of public access. Instead of using a traditional web server like Apache or Nginx, I used S3's built-in static website hosting feature, which is cost-effective and requires minimal configuration.

The architecture was straightforward: I created a dedicated S3 bucket, enabled static website hosting, uploaded the website files (index.html), and configured a bucket policy to allow public read access. I also explored access-control mechanisms, updated the website assets, and documented the risks associated with public buckets. Later, I investigated how a production architecture could use CloudFront with Origin Access Identity (OAI) to restrict access and improve performance.

2. Implementation Steps

3. Key AWS Concepts Learned

- **S3 Static Website Hosting:** S3 can serve HTML, CSS, and JavaScript files directly without a web server.
- **Bucket Policies:** JSON documents that define access permissions (e.g., s3:GetObject for public read).
- **Block Public Access:** A master switch that overrides all public access settings.

- **Origin Access Identity (OAI):** A CloudFront feature that allows private S3 buckets to serve content via CDN without making them public.
 - **Object Metadata:** System-defined metadata like Content-Type and Last-Modified help manage files.
 - **Versioning:** Keeping multiple versions of an object to protect against accidental deletions or overwrites.
 - **Cost Factors:** S3 charges for storage, requests, and data transfer out to the internet.
-

4. Challenges Faced

1. **Website Access Denied:** Initially, the website returned an "Access Denied" error because Block Public Access was enabled. I had to disable it and add a proper bucket policy.
 2. **Bucket Policy Syntax Error:** When adding the bucket policy, I initially had a syntax error (missing quotes or incorrect ARN). I fixed it by validating the JSON.
 3. **Understanding Metadata:** I was confused about the difference between system metadata and user-defined metadata, but exploring the S3 console clarified it.
 4. **Cleanup Consideration:** Since the website was working, I decided to keep the bucket active for demonstration purposes rather than deleting it immediately.
-

5. Solutions Implemented

1. **Public Access Fix:** I disabled Block Public Access for the bucket and added a strict bucket policy allowing only s3:GetObject for everyone.
2. **Policy Validation:** I used the AWS Policy Simulator to test my policy before saving it, which caught the syntax error early.

3. **Metadata Exploration:** I used the S3 console's "Properties" tab to view and understand object metadata.
 4. **Documentation:** I kept a detailed README to document all steps, decisions, and security considerations.
-

6. Security Considerations

1. **Least Privilege for Public Access:** Even though the bucket is "public" for the website, the policy only allows s3:GetObject. No write or delete permissions are granted to the public.
 2. **No Sensitive Data:** I ensured that no sensitive or confidential data was stored in the bucket, as it was publicly accessible.
 3. **Block Public Access Override:** I used the bucket policy for granular control and kept the Block Public Access settings consistent.
 4. **Production Recommendation:** For production, I recommended using CloudFront with OAI to keep the bucket private while still serving content publicly via CDN.
-

7. Results and Observations

Allowed Operations

Operation	URL	Result
Website Access	http://task8-nidhi-website.s3-website.us-east-1.amazonaws.com	Page loaded successfully
Updated Content	Same URL after upload	Version 2.0 displayed with timestamp

Observations

- Static website hosting on S3 is incredibly simple and fast to set up.
- Updating assets is as easy as re-uploading the file.

- Exploring metadata helped me understand how S3 stores

Improvement	Benefit
CloudFront + OAI	Restricts direct S3 access, adds caching, and provides HTTPS.
Bucket Versioning	Protects against accidental deletions or overwrites.
Lifecycle Policies	Automates storage tier transitions to save costs.
Custom Domain + SSL	Makes the website look professional with a custom domain and HTTPS.
Enable CloudWatch Metrics	Monitors bucket activity and alerts on unusual access patterns.
Use AWS WAF	Protects against malicious requests and DDoS attacks.
Enable S3 Server Access Logging	Provides detailed logs for auditing and compliance.

additional information about objects.

- Public buckets are convenient for static websites but should be used with caution and clear policies.

8. Possible Improvements for Production

Conclusion

This task provided practical experience in hosting a static website on S3, from setting up the bucket to updating assets and understanding security risks. The key takeaway is that S3 is a powerful and cost-effective

solution for hosting static content, but it requires careful attention to access-control mechanisms and public exposure.